

Dossiê de Pesquisa: Extração de Documentos com IA Generativa Multimodal

Lucas Henrique Nogueira

Junho de 2026

Resumo

Este dossiê avalia alternativas para reduzir latência, custo e complexidade em um pipeline de extração documental baseado em LLM. A solução atual descrita no desafio utiliza duas inferências: uma para extração do conteúdo e outra para formatação da resposta em JSON. Foram analisadas abordagens multimodais, pipelines híbridos e modelos especializados em Document AI, como Donut, LayoutLMv3 e Nougat. A recomendação final é uma arquitetura multimodal de chamada única, validada em uma POC com três casos: CNH, fatura de energia e documento técnico extenso. Os experimentos indicaram viabilidade para documentos curtos e médios, com necessidade de otimizações adicionais para PDFs extensos.

Sumário

1	Introdução	3
2	Metodologia	3
3	Técnicas e Modelos Avaliados	4
3.1	Baseline: pipeline atual com duas inferências LLM	4
3.2	Abordagem 1: modelo multimodal generalista com chamada única	5
3.3	Abordagem 2: modelos especializados para documentos estruturados	5
3.3.1	Donut	5
3.3.2	LayoutLMv3	6
3.4	Abordagem 3: modelo especializado para PDFs científicos	6
3.5	Abordagens consideradas, mas não priorizadas	6
3.6	Síntese da shortlist	7
4	Resultados e Experimentos	7
4.1	Resultados da POC	7
4.2	Validação estrutural da POC	8
4.3	Resultados reportados: Donut	8
4.4	Resultados reportados: LayoutLMv3	9
4.5	Resultados reportados: Nougat	10
4.6	Comparação consolidada	10
5	Discussão dos Trade-offs	11
5.1	Leitura comparativa das abordagens	11
5.2	Latência versus qualidade	12
5.3	Especialização versus generalidade	12
5.4	Custo de integração versus custo de inferência	12
5.5	Riscos e mitigações	13
6	Conclusão e Recomendação	13
7	Análise de Viabilidade de Integração	14
7.1	Arquitetura proposta	14
7.2	Infraestrutura necessária	14
7.3	Estimativa de custo	15
7.4	Escalabilidade	15
7.5	Riscos técnicos	16
7.6	Segurança e privacidade	16

1 Introdução

A extração automática de dados de documentos é uma tarefa comum em produtos que precisam processar identificações, faturas, contratos, formulários e relatórios técnicos.

No cenário analisado, a API atual utiliza um pipeline com duas inferências LLM: a primeira para interpretar o documento e extrair as informações relevantes, e a segunda para formatar a resposta no schema JSON esperado. Embora essa abordagem funcione, ela aumenta a latência, o custo por documento e a complexidade operacional do sistema. Além disso, documentos com layouts densos, tabelas, gráficos e imagens não textuais tornam o problema mais difícil, pois a informação relevante nem sempre está apenas no texto extraído.

O problema se torna mais evidente nos três casos avaliados neste trabalho: uma CNH, uma fatura de energia e um documento técnico extenso. Esses casos exigem capacidades diferentes do modelo, incluindo extração de campos específicos, interpretação de layout, leitura de tabelas e preservação da estrutura textual de PDFs longos.

O objetivo deste dossiê é avaliar alternativas para simplificar esse pipeline, reduzindo o número de chamadas ao modelo sem perder qualidade na extração. Para isso, foram analisadas abordagens multimodais, pipelines híbridos e modelos especializados em Document AI. A hipótese investigada é que uma arquitetura multimodal de chamada única pode produzir diretamente a saída final esperada, em JSON ou Markdown, dependendo do tipo de documento.

2 Metodologia

A pesquisa combinou análise bibliográfica, documentação técnica e uma POC prática. Como não havia acesso ao ambiente produtivo da API descrita no desafio, o pipeline atual de duas inferências foi tratado como baseline conceitual. A validação empírica foi feita apenas sobre a arquitetura proposta, enquanto as demais alternativas foram comparadas com base em benchmarks e resultados reportados na literatura.

Foram consideradas quatro abordagens principais: o pipeline atual com duas chamadas LLM, uma arquitetura multimodal de chamada única, um pipeline híbrido com extração local seguida de LLM, e modelos especializados em Document AI. A análise utilizou como fontes a documentação oficial da API Gemini, artigos científicos sobre Donut, LayoutLMv3 e Nougat, além de documentações de ferramentas de processamento de PDF.

Os critérios de comparação foram definidos a partir dos requisitos do desafio:

- latência média por documento;
- custo estimado por inferência;
- qualidade da extração;
- suporte a imagens, tabelas, gráficos e layouts complexos;
- complexidade de integração;
- necessidade de infraestrutura própria;
- aderência aos três casos: CNH, fatura de energia e documento técnico extenso.

A POC implementada utiliza uma única chamada multimodal ao modelo Gemini. Para CNH e fatura, a saída esperada é JSON estruturado. Para o documento técnico extenso, a saída esperada é Markdown, preservando a organização do conteúdo.

O código-fonte, os scripts de execução e os artefatos da POC estão disponíveis no repositório do projeto [7].

Os experimentos mediram a latência total da POC nos três documentos de teste. Cada caso foi executado duas vezes, registrando tamanho do arquivo, tempo total de execução, arquivo de saída e status da chamada. Como não havia um ground truth anotado para os documentos utilizados, a acurácia da POC foi avaliada qualitativamente, enquanto métricas quantitativas de qualidade foram obtidas a partir dos benchmarks publicados nos artigos analisados.

Algumas alternativas foram descartadas por não se alinharem ao objetivo principal da pesquisa. Pipelines baseados apenas em OCR clássico foram descartados porque extraem texto, mas não resolvem bem interpretação visual, gráficos, imagens e organização semântica de layouts complexos. Serviços comerciais especializados de Document AI, como soluções prontas de OCR e extração documental, também não foram priorizados porque mudariam o foco da análise para comparação entre produtos fechados, enquanto o desafio pedia investigar alternativas técnicas ao pipeline LLM atual. Por fim, abordagens locais com constrained decoding ou modelos pequenos apenas textuais foram consideradas pouco adequadas, pois não cobrem diretamente o requisito multimodal dos três casos avaliados.

Claude (Anthropic - Sonnet 4.6) foi utilizado como principal ferramenta de apoio durante o desenvolvimento desse trabalho, auxiliando na organização da pesquisa, estruturação do código da POC e revisão de partes do texto. As decisões de arquitetura, critérios de comparação, análise dos benchmarks e a recomendação final foram definidas a partir da interpretação dos experimentos próprios e das fontes técnicas consultadas.

3 Técnicas e Modelos Avaliados

A partir da pesquisa, selecionei três abordagens tecnicamente viáveis para análise mais detalhada: modelos multimodais generalistas com chamada única, modelos especializados em documentos estruturados e modelos especializados em conversão de PDFs científicos. O pipeline atual de duas inferências foi mantido como baseline conceitual, pois representa o ponto de partida descrito no desafio.

3.1 Baseline: pipeline atual com duas inferências LLM

O pipeline atual descrito no desafio utiliza duas chamadas ao modelo. Na API de referência, o cliente informa a URL do arquivo e o identificador do layout previamente configurado, que define quais campos devem ser extraídos. A primeira chamada interpreta o documento e extrai as informações relevantes; a segunda recebe essa saída intermediária e formata o resultado no schema JSON esperado.

Essa separação pode facilitar o controle da etapa de formatação, mas tem duas consequências diretas: aumenta a latência total e duplica parte do custo de inferência. Além disso, erros da primeira etapa podem ser propagados para a segunda. Por não haver acesso ao ambiente da API, esse pipeline foi tratado como baseline conceitual, e não como baseline experimental.

3.2 Abordagem 1: modelo multimodal generalista com chamada única

A primeira abordagem avaliada foi o uso de um modelo multimodal generalista capaz de receber diretamente imagens ou PDFs e retornar a saída final em uma única chamada. Essa foi a abordagem implementada na POC, usando Gemini com prompts específicos por tipo documental.

Essa solução é aderente ao problema porque os três casos do desafio exigem leitura de campos, interpretação de layout e conversão de PDF técnico extenso. A documentação do Gemini indica suporte a documentos com texto, imagens, tabelas e elementos visuais [2], além de mecanismos para orientar saídas estruturadas em JSON [4]. Na POC, essa ideia foi implementada com prompts restritivos e validação local das respostas; em produção, a próxima etapa seria usar schema nativo da API.

A diferença ficou nos prompts e no formato de saída: JSON para CNH e fatura de energia, e Markdown para o documento técnico extenso. Essa decisão reduz a necessidade de componentes especializados, como OCR separado, parser de layout ou fine-tuning por tipo documental.

Sem GPU própria ou dataset anotado, a integração se resume a uma chamada de API com prompt e formato definidos. A contrapartida é a dependência de fornecedor externo e a latência variável em documentos grandes.

3.3 Abordagem 2: modelos especializados para documentos estruturados

A segunda abordagem analisada foi o uso de modelos especializados em Visual Document Understanding e Document AI. Dentro dessa linha, os dois modelos mais relevantes foram Donut e LayoutLMv3.

3.3.1 Donut

O Donut propõe uma arquitetura OCR-free para entendimento de documentos visuais [6]. O ponto mais importante do artigo é a crítica ao pipeline tradicional de Document AI: primeiro detectar texto, depois reconhecer caracteres via OCR e, por fim, passar o resultado para um modelo de entendimento. Segundo os autores, esse desenho pode gerar alto custo computacional, baixa flexibilidade entre idiomas e domínios, além de propagação de erros de OCR para as etapas seguintes.

A proposta do Donut é tratar o documento como imagem e gerar diretamente a saída textual ou estruturada com uma arquitetura encoder-decoder, alinhando-se ao objetivo de reduzir etapas, custo e latência.

Nos resultados reportados, o Donut obteve no benchmark CORD, voltado a recibos, tempo de 1.2 segundos por imagem, F1 de 84.1 e acurácia de 90.9. No DocVQA, reportou ANLS de 67.5 com tempo de 782 ms. Esses números mostram que a abordagem OCR-free é competitiva, principalmente considerando que não depende de um OCR externo.

Apesar disso, o Donut não foi escolhido como solução principal. O motivo é pragmático: o artigo reporta pré-treinamento com 2M documentos sintéticos, 11M imagens do IIT-CDIP, 200K steps e 64 GPUs A100 [6]. Para uma POC e para uma primeira integração de produto, isso aumenta o custo inicial, a complexidade de manutenção e a necessidade de adaptação por domínio.

3.3.2 LayoutLMv3

O LayoutLMv3 é um modelo multimodal especializado em Document AI, combinando texto, layout e imagem [5]. Diferente do Donut, ele não elimina a necessidade de informações textuais: o artigo usa OCR annotations no CORD e Microsoft Read API em tarefas como RVL-CDIP e DocVQA, indicando dependência de pré-processamento textual em parte dos cenários avaliados.

Seu diferencial técnico está no pré-treinamento unificado de texto e imagem, com objetivos como Masked Language Modeling, Masked Image Modeling e Word-Patch Alignment. Esses objetivos buscam alinhar palavras, layout e regiões visuais, tornando o modelo forte para formulários, recibos, classificação documental, VQA e análise de layout.

Nos benchmarks, o LayoutLMv3 Large reporta F1 de 92.08 no FUNSD, F1 de 97.46 no CORD e ANLS de 83.37 no DocVQA. Esses resultados indicam que o modelo é uma alternativa robusta para documentos estruturados.

Mesmo assim, ele não foi escolhido como recomendação principal. Em produção, seria necessário montar um pipeline com OCR ou extração de texto, coordenadas de layout, adaptação ao domínio da empresa e possivelmente fine-tuning. Portanto, é uma solução tecnicamente forte, mas menos simples de integrar do que uma API multimodal generalista.

3.4 Abordagem 3: modelo especializado para PDFs científicos

A terceira abordagem avaliada foi o Nougat, modelo especializado em converter documentos acadêmicos para linguagem de marcação [1]. Ele é particularmente relevante para o documento técnico extenso, pois o desafio pede preservação de organização textual, tabelas, gráficos e imagens.

O artigo parte de uma limitação dos PDFs científicos: embora sejam legíveis para humanos, eles frequentemente perdem informação semântica quando convertidos para texto simples, especialmente em expressões matemáticas, tabelas e estruturas de formatação. O Nougat trata cada página como imagem e gera uma representação em markup, seguindo uma linha próxima ao Donut, mas especializada em documentos acadêmicos.

Nos resultados no conjunto de teste arXiv, o Nougat Base apresenta desempenho forte na conversão de documentos científicos para markup, especialmente em texto puro. Ao mesmo tempo, os resultados indicam que tabelas e expressões matemáticas continuam sendo componentes mais difíceis do problema.

O Nougat é uma alternativa forte para papers e relatórios científicos. No entanto, ele não resolve bem o problema completo do desafio, pois não foi projetado para documentos como CNH e faturas de energia. Por isso, foi considerado uma opção especializada para evolução futura do caso de documentos extensos, e não como solução geral.

3.5 Abordagens consideradas, mas não priorizadas

Também foi considerada uma solução híbrida com extração local de texto, por exemplo usando PyMuPDF [8], seguida de LLM para estruturação. Essa abordagem pode ser eficiente em PDFs digitais simples, pois reduz o conteúdo enviado ao modelo. Porém, foi descartada como solução principal porque não cobre bem documentos escaneados, imagens de documentos, faturas com layout visual complexo e elementos gráficos, nos quais a informação relevante depende também da posição e da organização visual.

3.6 Síntese da shortlist

A Tabela 1 resume o papel de cada abordagem na análise.

Tabela 1: Shortlist de técnicas e modelos avaliados.

Abordagem	Ponto forte	Limitação principal	Papel na recomendação
Gemini multimodal single-call	Baixa complexidade de integração e suporte direto a imagens e PDFs	Dependência de API externa e latência alta em PDFs extensos	Recomendação principal e base da POC
Donut / LayoutLMv3	Bons resultados em benchmarks de documentos estruturados, como CORD, FUNSD e DocVQA	Exigem pipeline especializado, infraestrutura e/ou fine-tuning	Alternativas futuras para domínios estáveis e alto volume
Nougat	Forte para conversão de PDFs científicos em markup	Escopo restrito a documentos acadêmicos/científicos	Alternativa futura para documentos técnicos extensos

Com base nessa análise, a abordagem multimodal de chamada única foi escolhida para a POC por oferecer o melhor equilíbrio entre cobertura dos três casos, simplicidade de integração e viabilidade de implementação em curto prazo. Os modelos especializados apresentaram resultados superiores em benchmarks específicos, mas com maior custo de manutenção e menor generalidade.

4 Resultados e Experimentos

Esta seção reúne dois tipos de resultado: os experimentos executados na POC e os resultados reportados nos artigos analisados. A POC foi avaliada em latência, enquanto os modelos especializados foram avaliados a partir dos benchmarks publicados pelos próprios autores.

4.1 Resultados da POC

A POC foi implementada com uma chamada única ao modelo multimodal Gemini. Cada documento foi enviado junto com um prompt específico para o tipo documental. A saída esperada foi JSON para CNH e fatura de energia, e Markdown para o documento técnico extenso.

Cada caso foi executado duas vezes. A latência foi medida do início da execução do script até a escrita do arquivo de saída. Portanto, o tempo inclui leitura do arquivo, preparação do envio, chamada ao modelo, processamento da resposta e gravação do resultado. Os resultados são apresentados na Tabela 2.

Tabela 2: Latência observada na POC.

Caso	Arquivo	Tamanho	Média	Mín.	Máx.
CNH	cnh_sample.jpeg	0.08 MB	6.55s	6.40s	6.71s
Fatura	energy_invoice.jpg	0.08 MB	25.09s	20.39s	29.79s
Paper	claude_paper.pdf	26.93 MB	208.30s	151.09s	265.51s

Os resultados indicam que a abordagem é viável para documentos curtos e médios. A CNH foi processada em menos de 7 segundos em média. A fatura, apesar de ter tamanho semelhante, teve latência maior, o que sugere impacto da densidade visual e da quantidade de campos. O paper apresentou latência muito superior, principalmente por ser um PDF extenso de 26.93 MB e exigir geração de uma saída longa em Markdown.

Como os documentos usados na POC não possuíam ground truth anotado, não foi calculado F1 próprio. A avaliação quantitativa de qualidade foi complementada com os benchmarks publicados nos artigos Donut, LayoutLMv3 e Nougat.

4.2 Validação estrutural da POC

Além da latência, foi feita uma validação estrutural das saídas geradas pela POC. Como os documentos utilizados não possuem ground truth anotado, não foram calculadas métricas supervisionadas como F1, precisão ou recall para a implementação própria. Essas métricas foram usadas apenas na análise dos benchmarks reportados nos artigos.

Na POC, foram observados três critérios simples, resumidos na Tabela 3:

- se a saída foi gerada no formato esperado;
- se os campos obrigatórios estavam presentes;
- se, nos casos de fatura e documento extenso, a saída preservou alguma estrutura de layout, seções, tabelas ou figuras.

Tabela 3: Validação estrutural das saídas da POC.

Caso	Formato esperado	Validade do formato	Campos obrigatórios	Observação estrutural
CNH	JSON	2/2 execuções válidas	6/6 campos presentes	Saída estruturada com nome, CPF, datas e filiação
Fatura	JSON	2/2 execuções válidas	7/7 campos presentes	Saída incluiu seções de layout e blocos tabulares detectados
Paper	Markdown	2/2 execuções válidas	Não aplicável	Saída preservou títulos, seções, tabelas e referências a figuras

Essa validação não mede acurácia semântica completa. Para isso, seria necessário construir uma base anotada com os valores esperados por campo e comparar automaticamente as respostas do modelo com esse ground truth.

4.3 Resultados reportados: Donut

O Donut é relevante porque avalia uma arquitetura OCR-free para documentos visuais. O artigo compara o modelo com abordagens dependentes de OCR em tarefas como CORD, Ticket, Business Card, Receipt e DocVQA. A Tabela 4 resume os resultados de extração estruturada, com tempo, F1 e acurácia para diferentes domínios. O ponto principal é que o Donut aparece sem dependência de OCR externo nessa comparação [6].

Tabela 4: Resultados do Donut em tarefas de extração estruturada reportados no artigo.

Dataset	Tempo	F1	Acurácia	Observação
CORD	1.2s	84.1	90.9	Recibos
Ticket	0.6s	94.1	98.7	Bilhetes/tickets
Business Card	1.4s	57.8	84.4	Cartões de visita
Receipt	1.9s	78.6	88.6	Recibos industriais

A Tabela 5 mostra que, no DocVQA, o Donut reporta ANLS geral de 67.5 com tempo de 782 ms e 176M parâmetros. Esse resultado é inferior ao LayoutLMv2 em ANLS geral, mas o artigo destaca que o Donut não depende de OCR e apresentou bom resultado em documentos manuscritos, com ANLS de 72.1 nessa categoria [6].

Tabela 5: Resultado do Donut em DocVQA reportado no artigo.

Modelo	OCR externo	Parâmetros	Tempo	ANLS geral	ANLS manuscrito
Donut	Não	176M	782ms	67.5	72.1
LayoutLMv2	Sim	200M + α	1610ms	78.1	–
LayoutLMv2-Large-QG	Sim	390M + α	1698ms	86.7	67.3

A leitura principal é que o Donut reduz a dependência de OCR e apresenta boa velocidade, mas não domina todos os benchmarks. Para o contexto deste desafio, ele é uma alternativa tecnicamente interessante, mas exigiria integração e adaptação específicas.

4.4 Resultados reportados: LayoutLMv3

A Tabela 6 mostra que o LayoutLMv3 apresenta resultados fortes em benchmarks clássicos de Document AI. A tabela principal do artigo compara modelos em FUNSD, CORD, RVL-CDIP e DocVQA. As métricas são F1 para FUNSD e CORD, acurácia para RVL-CDIP e ANLS para DocVQA [5].

Tabela 6: Resultados do LayoutLMv3 reportados no artigo.

Modelo	Parâmetros	FUNSD F1	CORD F1	RVL-CDIP Acc.	DocVQA ANLS
LayoutLMv3 Base	133M	90.29	96.56	95.44	78.76
LayoutLMv3 Large	368M	92.08	97.46	95.93	83.37
LayoutLMv2 Base	200M	82.76	94.95	95.25	78.08
LayoutLMv2 Large	426M	84.20	96.01	95.64	83.48
DocFormer Base	183M	83.34	96.33	96.17	–
DocFormer Large	536M	84.55	96.99	95.50	–

A Tabela 7 resume o desempenho em análise de layout no PubLayNet. Nessa tarefa, o LayoutLMv3 Base atinge mAP geral de 95.1, com bons resultados em texto, títulos, listas, tabelas e figuras [5].

Tabela 7: LayoutLMv3 Base em análise de layout no PubLayNet.

Modelo	Texto	Título	Lista	Tabela	Figura	Geral
LayoutLMv3 Base	94.5	90.6	95.5	97.9	97.0	95.1
DiT Base	94.4	88.9	94.8	97.6	96.9	94.5
UDoc	93.9	88.5	93.7	97.3	96.4	93.9

O ponto importante é que o LayoutLMv3 tem resultados fortes, principalmente em formulários, recibos e layout. Porém, esses números não incluem uma latência de inferência diretamente comparável à POC. Além disso, o próprio artigo usa OCR annotations no CORD e Microsoft Read API em parte dos experimentos, indicando que a solução depende de pré-processamento textual/layout em vários cenários.

4.5 Resultados reportados: Nougat

O Nougat foi analisado por ser especializado em documentos acadêmicos, com resultados resumidos na Tabela 8. O artigo avalia conversão de PDFs científicos para linguagem de marcação usando o conjunto de teste arXiv. As métricas reportadas incluem edit distance, BLEU, METEOR, precisão, recall e F1 [1].

Tabela 8: Resultados do Nougat e baselines no conjunto de teste arXiv.

Método	Modalidade	Edit dist.	BLEU	METEOR	Precisão	Recall	F1
PDF embutido	All	0.255	65.8	82.1	77.1	81.4	79.2
GROBID	All	0.312	55.6	71.9	74.0	72.1	73.0
GROBID	Tables	0.626	25.1	64.5	61.4	80.7	69.7
GROBID + LaTeX OCR	Plain text	0.363	57.4	69.2	82.1	70.5	75.9
GROBID + LaTeX OCR	Math	0.727	0.3	5.0	11.0	8.6	9.7
Nougat Small	All	0.073	88.9	92.8	93.6	92.2	92.9
Nougat Small	Tables	0.220	68.5	78.6	75.0	79.8	77.3
Nougat Small	Plain text	0.058	91.0	94.3	96.1	95.3	95.7
Nougat Small	Math	0.117	56.0	74.7	77.1	76.8	76.9
Nougat Base	All	0.071	89.1	93.0	93.5	92.8	93.1
Nougat Base	Tables	0.211	69.7	79.1	75.4	80.7	78.0
Nougat Base	Plain text	0.058	91.2	94.6	96.2	95.3	95.7
Nougat Base	Math	0.128	56.9	75.4	76.5	76.6	76.5

O Nougat Base tem 350M parâmetros e apresenta ganho claro sobre PDF embutido e GROBID no resultado geral. O F1 geral sobe de 79.2 no texto extraído do PDF para 93.1 no Nougat Base. Em texto puro, o F1 chega a 95.7. Porém, tabelas e matemática continuam sendo mais difíceis, com F1 de 78.0 e 76.5, respectivamente.

Esses resultados são muito relevantes para o caso do documento técnico extenso, mas não cobrem CNH nem fatura. Por isso, o Nougat foi tratado como alternativa especializada, não como solução geral.

4.6 Comparação consolidada

A Tabela 9 resume os resultados mais importantes para a decisão técnica. Os tempos da POC foram medidos no experimento próprio. Os demais valores vêm dos artigos e não são diretamente comparáveis, pois foram obtidos em datasets, ambientes e tarefas diferentes.

Tabela 9: Comparação consolidada entre POC e benchmarks de literatura.

Abordagem	Evidência	Melhores resultados	Latência	Leitura para o desafio
Gemini single-call	POC própria	Executou CNH, fatura e paper com a mesma arquitetura	6.55s, 25.09s e 208.30s	Melhor opção inicial
Donut	Artigo	CORD: F1 84.1 / Acc. 90.9; DocVQA: ANLS 67.5	1.2s no CORD; 782ms no DocVQA	Forte sem OCR; exige adaptação
LayoutLMv3	Artigo	FUNSD F1 92.08; CORD F1 97.46; DocVQA ANLS 83.37	Não reportada de forma comparável	Alto desempenho; pipeline complexo
Nougat Base	Artigo	arXiv: F1 geral 93.1; texto 95.7; tabelas 78.0; matemática 76.5	Não reportada de forma comparável	Excelente para papers; escopo restrito

A conclusão dos resultados é que os modelos especializados são fortes nos seus domínios, mas não resolvem igualmente bem todos os casos do desafio. O LayoutLMv3 apresenta resultados muito fortes em formulários, recibos e análise de layout; o Nougat é mais adequado para documentos científicos; e o Donut é uma alternativa interessante para remover OCR do pipeline. Porém, a POC com modelo multimodal generalista é a opção mais pragmática para cobrir os três casos com menor esforço de integração.

5 Discussão dos Trade-offs

Os resultados mostram que a decisão não depende apenas do maior F1 ou do menor tempo reportado em um benchmark. Os artigos analisados avaliam tarefas, datasets e ambientes diferentes, portanto os números não são diretamente comparáveis com a POC. Mesmo assim, eles ajudam a entender os trade-offs entre especialização, custo de integração, latência e cobertura dos casos do desafio.

5.1 Leitura comparativa das abordagens

A Tabela 10 resume a interpretação das principais alternativas avaliadas.

Tabela 10: Trade-offs práticos das abordagens avaliadas.

Abordagem	Ponto forte	Limitação principal	Impacto no desafio
Gemini multimodal single-call	Integração simples, aceita imagens e PDFs e produz JSON ou Markdown diretamente	Latência variável em documentos extensos e dependência de API externa	Melhor opção para POC e primeira integração, pois cobre CNH, fatura e paper com a mesma arquitetura
Donut	Arquitetura OCR-free, com boa velocidade em documentos visuais	Exige adaptação, infraestrutura própria e pode não superar modelos com OCR em todos os cenários	Forte candidato para documentos visuais estruturados em alto volume, mas não é a opção mais simples para adoção inicial
LayoutLMv3	Alto desempenho em formulários, recibos, DocVQA e análise de layout	Depende de texto/layout extraído, OCR ou anotações, além de possível fine-tuning	Excelente para domínios estruturados, mas aumenta a complexidade do pipeline
Nougat	Muito forte para converter PDFs científicos em markup	Escopo restrito a documentos acadêmicos; tabelas e matemática ainda são partes difíceis	Útil para o caso do paper técnico, mas não resolve CNH e fatura
Híbrido local + LLM	Pode reduzir custo em PDFs digitais com texto selecionável	Perde informação visual em documentos escaneados, imagens, faturas densas e gráficos	Boa otimização futura, mas não cobre de forma uniforme os três casos obrigatórios

5.2 Latência versus qualidade

O Donut mostra um trade-off importante. No CORD, o modelo reporta F1 de 84.1 e acurácia de 90.9 com tempo de 1.2 segundos por imagem. No DocVQA, reporta ANLS de 67.5 com tempo de 782 ms [6]. Esses resultados indicam boa velocidade e simplicidade conceitual, já que o modelo não depende de OCR externo.

Por outro lado, o próprio resultado em DocVQA mostra que eliminar OCR não garante sempre o melhor desempenho. Modelos dependentes de OCR, como LayoutLMv2, reportam ANLS maior nesse benchmark, embora com maior tempo e com o custo adicional do OCR. Portanto, o Donut reduz uma etapa clássica do pipeline, mas não elimina a necessidade de avaliar qualidade por domínio.

O LayoutLMv3 representa o outro lado desse trade-off. Ele reporta resultados muito fortes em tarefas estruturadas: F1 de 92.08 no FUNSD, F1 de 97.46 no CORD e ANLS de 83.37 no DocVQA na versão Large [5]. Esses números são superiores aos do Donut em várias tarefas, mas vêm acompanhados de uma arquitetura mais especializada, que combina texto, layout e imagem.

Na POC, a abordagem Gemini single-call não foi avaliada com F1 por falta de ground truth anotado. A evidência prática foi outra: a mesma arquitetura processou os três tipos de documento exigidos. Assim, a vantagem observada não foi superioridade em benchmark, mas cobertura e simplicidade de integração.

5.3 Especialização versus generalidade

Os modelos especializados analisados apresentam melhor desempenho quando o domínio do documento coincide com o domínio de treinamento e avaliação. Porém, o desafio combina três tipos documentais distintos: CNH, fatura de energia e documento técnico extenso. Nesse cenário, a generalidade passa a ter valor de produto. Por isso, a solução multimodal single-call foi priorizada como primeira evolução: ela não maximiza o desempenho em um domínio específico, mas cobre todos os casos obrigatórios com menor complexidade de integração.

5.4 Custo de integração versus custo de inferência

A arquitetura single-call tende a reduzir custo em relação ao pipeline atual porque elimina a segunda chamada LLM usada apenas para formatação. Em uma API comercial, isso reduz o número de inferências por documento e simplifica o fluxo de execução. O custo passa a depender principalmente dos tokens de entrada e saída da chamada multimodal [3].

Modelos especializados deslocam o custo de API para infraestrutura própria: ambiente de inferência, versionamento, monitoramento e, em geral, fine-tuning. Esse custo não aparece nas métricas de F1, mas é decisivo para uma adoção em produto — especialmente quando o domínio documental é variado como neste desafio.

5.5 Riscos e mitigações

A abordagem recomendada também possui riscos. O primeiro é a variabilidade da saída. Como a POC utiliza um modelo generativo, existe a possibilidade de respostas fora do schema esperado. Esse risco foi reduzido com prompts específicos por tipo documental e validação das saídas JSON.

O segundo risco é a latência em documentos extensos. Os experimentos indicaram que processar PDFs grandes em uma única chamada pode não ser adequado em produção síncrona. Uma mitigação seria processar documentos extensos por páginas ou seções, consolidando o resultado posteriormente.

O terceiro risco é a dependência de fornecedor externo. A solução baseada em API reduz esforço de infraestrutura, mas cria dependência de preço, disponibilidade, limites de uso e mudanças no modelo. Para reduzir esse risco, a implementação deve isolar o cliente LLM em uma camada própria e permitir fallback futuro para outros provedores ou modelos locais.

6 Conclusão e Recomendação

A recomendação principal deste dossiê é substituir o pipeline atual de duas inferências por uma arquitetura multimodal de chamada única. Nessa abordagem, o documento é enviado diretamente ao modelo junto com um prompt específico para o tipo documental, e a resposta já é produzida no formato final esperado: JSON para documentos estruturados e Markdown para documentos extensos.

Essa recomendação não assume que modelos multimodais generalistas sejam sempre superiores aos especializados. Os artigos analisados mostram que Donut, LayoutLMv3 e Nougat apresentam desempenho forte em seus domínios: documentos visuais OCR-free, documentos estruturados e PDFs científicos, respectivamente [6, 5, 1]. Porém, o desafio exige uma solução capaz de cobrir três tipos documentais distintos: CNH, fatura de energia e documento técnico extenso.

Nesse contexto, a arquitetura multimodal de chamada única apresentou o melhor equilíbrio entre cobertura, simplicidade de integração e viabilidade de adoção em curto prazo. A POC mostrou que a mesma arquitetura processou os três casos obrigatórios, com bom desempenho em documentos curtos e maior latência no documento técnico extenso.

O principal benefício esperado é a redução de complexidade do pipeline. Ao remover a segunda inferência dedicada apenas à formatação, a solução reduz o número de chamadas ao modelo, diminui a superfície de falha e simplifica a manutenção, sem exigir inicialmente treinamento ou hospedagem de modelos especializados.

A principal limitação observada está nos documentos extensos. O resultado do paper indica que processar PDFs grandes em uma única chamada pode não ser adequado para fluxos síncronos de produção. Para esse caso, recomenda-se evoluir a arquitetura com processamento por páginas ou seções, seguido de consolidação posterior.

Portanto, recomenda-se adotar a arquitetura multimodal de chamada única como primeira evolução do produto, mantendo Donut, LayoutLMv3 e Nougat como alternativas futuras para cenários de alto volume, domínios estáveis ou necessidades específicas de acurácia. A estratégia mais adequada é começar com uma solução simples e generalista, medir custo e qualidade em produção, e só então decidir se o ganho de modelos especializados justifica a complexidade adicional.

7 Análise de Viabilidade de Integração

A arquitetura proposta funciona como uma camada intermediária entre a aplicação e o modelo multimodal: recebe o documento, seleciona o prompt adequado, envia o arquivo ao modelo e valida a resposta antes de retorná-la ao sistema consumidor.

7.1 Arquitetura proposta

A integração pode ser organizada em cinco componentes:

1. **Entrada documental:** recebe arquivos PDF ou imagem e registra metadados básicos, como tipo documental, tamanho do arquivo e identificador da requisição.
2. **Roteamento por tipo de documento:** seleciona o prompt e o formato de saída esperado. CNH e fatura seguem para JSON; documentos técnicos extensos seguem para Markdown.
3. **Cliente multimodal:** encapsula a chamada ao modelo Gemini, incluindo tratamento de erro, tentativas e controle de limites de uso.
4. **Validação da resposta:** verifica se o JSON é válido nos casos estruturados e se o Markdown possui estrutura mínima nos documentos extensos.
5. **Observabilidade:** registra latência, status da chamada, tamanho do arquivo, falhas de parsing e custo estimado por documento.

Esse desenho mantém a simplicidade da POC, mas adiciona os elementos necessários para uso em produção. A separação entre roteamento, prompts, cliente do modelo e validação também facilita trocar o provedor multimodal no futuro, caso seja necessário.

7.2 Infraestrutura necessária

Para a solução recomendada, a inferência é executada por uma API externa. Portanto, a infraestrutura local é leve: um serviço backend, CPU convencional, memória suficiente para manipular arquivos temporários e armazenamento seguro para documentos durante o processamento. A infraestrutura local fica concentrada no serviço de orquestração: recebimento do arquivo, chamada à API, validação da resposta e registro de métricas. Para documentos curtos, como CNH e faturas simples, a execução pode ser síncrona. Para documentos extensos, recomenda-se processamento assíncrono com fila. Isso evita timeout na API principal e permite que o usuário acompanhe o status da extração.

7.3 Estimativa de custo

O custo da arquitetura recomendada depende principalmente do preço da API multimodal, do tamanho do documento, do prompt e do volume de tokens de saída. Na POC, o consumo real de tokens não foi registrado, portanto a estimativa abaixo deve ser interpretada como uma ordem de grandeza, não como medição definitiva.

Considerando o modelo `gemini-2.5-flash`, usado como padrão na POC, no modo Standard/Paid Tier, a precificação de referência é de US\$ 0,30 por 1 milhão de tokens de entrada em texto/imagem/vídeo e US\$ 2,50 por 1 milhão de tokens de saída, incluindo tokens de raciocínio [3]. Assim, o custo aproximado por documento pode ser estimado por:

$$C_{doc} = \frac{T_{in}}{10^6} \cdot 0.30 + \frac{T_{out}}{10^6} \cdot 2.50$$

onde T_{in} representa os tokens de entrada e T_{out} representa os tokens de saída.

Como exemplo, para documentos curtos como CNH ou fatura simples, assumindo aproximadamente 2.000 tokens de entrada e 800 tokens de saída, o custo estimado seria:

$$C_{curto} = \frac{2000}{10^6} \cdot 0.30 + \frac{800}{10^6} \cdot 2.50 \approx \text{US\$ } 0.0026$$

Ou seja, cerca de US\$ 0,0026 por documento, desconsiderando custos auxiliares de backend, armazenamento temporário e eventuais retentativas.

Para o documento técnico extenso, o custo tende a ser maior porque a saída em Markdown é longa. Assumindo, de forma conservadora, 80.000 tokens de entrada e 20.000 tokens de saída, o custo estimado seria:

$$C_{extenso} = \frac{80000}{10^6} \cdot 0.30 + \frac{20000}{10^6} \cdot 2.50 \approx \text{US\$ } 0.074$$

Esses valores reforçam o trade-off observado nos experimentos: documentos curtos são baratos e viáveis em fluxo síncrono, enquanto documentos extensos exigem controle de custo, processamento assíncrono e possivelmente divisão por páginas ou seções. Em produção, o custo real deve ser obtido via `usage_metadata` retornado pela API, que informa os tokens consumidos por chamada. Esses valores devem ser registrados por execução junto com tipo documental, latência e status da chamada.

7.4 Escalabilidade

A escalabilidade da solução depende principalmente de controle de concorrência, limites da API externa e estratégia de processamento para documentos longos.

Para documentos pequenos, a chamada síncrona é suficiente. Para documentos extensos, a arquitetura deve usar fila e workers. Nesse desenho, a API recebe o documento, cria uma tarefa, retorna um identificador e processa a extração em segundo plano. Essa estratégia permite controlar chamadas simultâneas, aplicar retentativas e evitar bloqueio da aplicação principal.

A POC mostrou esse ponto claramente: documentos curtos se adequam melhor a um fluxo simples, enquanto o documento técnico extenso exige tratamento assíncrono. Para produção, esse caso deve ser tratado com divisão por páginas ou seções e consolidação posterior.

7.5 Riscos técnicos

Os principais riscos de integração e suas mitigações são resumidos na Tabela 11.

Tabela 11: Riscos de integração e possíveis mitigações.

Risco	Impacto	Mitigação
Resposta fora do schema	Falha no consumo do JSON pela aplicação	Validação com schema, prompts restritivos e nova tentativa controlada
Latência alta em PDFs extensos	Timeout e pior experiência do usuário	Processamento assíncrono, divisão por páginas ou seções e consolidação posterior
Dependência de API externa	Risco de preço, limite de uso ou indisponibilidade	Isolar o cliente LLM e manter possibilidade de fallback para outro provedor ou modelo local
Custo elevado em alto volume	Aumento do custo operacional	Roteamento por tipo documental, cache, batch processing e uso de modelos mais baratos para casos simples
Exposição de dados sensíveis	Risco legal e reputacional	Minimização de logs, criptografia, controle de acesso e política clara de retenção

7.6 Segurança e privacidade

Os documentos processados podem conter CPF, nome, endereço, número de cliente e dados financeiros. Por isso, a integração deve evitar armazenamento desnecessário de arquivos e impedir que o conteúdo completo dos documentos apareça em logs.

As principais práticas recomendadas são: uso de HTTPS, armazenamento temporário com retenção curta, controle de acesso às credenciais da API, mascaramento de dados sensíveis em logs e registro apenas de metadados operacionais, como latência, tipo documental, status e tamanho do arquivo. Os documentos utilizados nos experimentos da POC são amostras genéricas sem dados pessoais reais.

Referências

- [1] Lukas Blecher, Guillem Cucurull, Thomas Scialom, and Robert Stojnic. Nougat: Neural Optical Understanding for Academic Documents. *arXiv preprint arXiv:2308.13418*, 2023.
- [2] Google AI for Developers. Gemini api: Document understanding. <https://ai.google.dev/gemini-api/docs/document-processing>, 2026. Accessed: 2026-06-28.
- [3] Google AI for Developers. Gemini api pricing. <https://ai.google.dev/gemini-api/docs/pricing>, 2026. Accessed: 2026-06-28.
- [4] Google AI for Developers. Gemini api: Structured outputs. <https://ai.google.dev/gemini-api/docs/structured-output>, 2026. Accessed: 2026-06-28.
- [5] Yupan Huang, Tengchao Lv, Lei Cui, Yutong Lu, and Furu Wei. LayoutLMv3: Pre-training for Document AI with Unified Text and Image Masking. *arXiv preprint arXiv:2204.08387*, 2022.
- [6] Geewook Kim, Teakgyu Hong, Moonbin Yim, Jeongyeon Nam, Jinyoung Park, Jinyeong Yim, Wonseok Hwang, Sangdoo Yun, Dongyoon Han, and Seunghyun Park. OCR-free Document Understanding Transformer. *arXiv preprint arXiv:2111.15664*, 2022.
- [7] Lucas Henrique Nogueira. genai-document-extraction-poc. <https://github.com/Lucas-Henriquee/genai-document-extraction-poc>, 2026. Accessed: 2026-06-28.
- [8] PyMuPDF. Pymupdf documentation: Text extraction. <https://pymupdf.readthedocs.io/en/latest/recipes-text.html>, 2026. Accessed: 2026-06-28.